

```

1 import tensorflow as tf
2 x = tf.constant([-111.0, -222.0, 0.0, 333.0, 444.5], dtype=tf.float32)
3 print(tf.nn.relu(x))
4 print(tf.nn.leaky_relu(x))

```

tf.Tensor([0. 0. 0. 333. 444.5], shape=(5,), dtype=float32)
tf.Tensor([-22.2 -44.4 0. 333. 444.5], shape=(5,), dtype=float32)

Figure 10: TensorFlow for neural networks

```

1 from tensorflow import keras
2 from keras.datasets import mnist
3 import matplotlib.pyplot as plt
4
5 (train_imgs, train_lbls), (test_imgs, test_lbls) = mnist.load_data()
6 print(train_imgs.ndim)
7 print(train_imgs.shape)
8 print(test_imgs.ndim)
9 print(test_imgs.shape)
10 sample = train_imgs[1234]
11 plt.imshow(sample, cmap=plt.cm.binary)
12 plt.show()

```

Figure 11: Our first data set

data fitting. Linear regression is very important because of its simplicity. The supervised learning paradigm of machine learning is actually another name for regression modelling. Thus, linear regression can be considered as an important machine learning strategy. This learning paradigm is often called statistical learning by statisticians. Both NumPy and SciPy offer functions for performing linear regression. Since linear regression is an important operation in machine learning, we will see examples involving both NumPy and SciPy. Now, consider the program shown in Figure 6.

The program uses NumPy to perform linear regression. First, let us try to understand the working of this program. Lines 1 and 2 import NumPy and Matplotlib. Lines 4 and 5 initialise the variables *a* and *b* for data generation. Line 6 uses the function *linspace()* to generate

evenly spaced numbers over the specified interval of 0 and 1. In this case 100 such numbers are generated. Line 7 generates values in the array *y* by using the values in the array *x*. The function *randn()* returns samples from the standard normal distribution. These values, together with the values provided by the variables *a*, *b*, and array *x*, are used to generate the values in the array *y*. Line 8 generates a scatter plot by using the values in the arrays *x* and *y*. Figure 7 shows this scatter plot. The hundred data points are marked in red. Line 9 uses the function *polyfit()* to perform a least squares polynomial fit. It is one of the techniques used to perform linear regression. The function *polyfit()* takes for processing the arrays *x* and *y*, and a third parameter (in this case 1), which denotes the degree of the fitting polynomial. The function then returns the polynomial coefficients

and they are stored in the array *p*. Line 10 uses the function *polyval()*, which evaluates a polynomial at specific values. These values are stored in the array *y_l*. Line 11 plots the regression line in blue (Figure 7). Finally, Line 12 displays all the plots. This regression line can be used for making predictions about the possible (*x*, *y*) data pairs.

Now, let us see how SciPy can be used to perform the same task. Consider the program shown in Figure 8.

Let us try to understand the working of the program. Lines 1 and 2 import the libraries NumPy and Matplotlib. Line 3 imports the *stats* module from the library SciPy. Lines 4 to 8 are the same as the previous program and perform similar tasks. Line 9 uses the function *linregress()* offered by the stats module of SciPy, which can calculate a linear least-squares regression for two sets of